PLANT DISEASE DETECTION USING CNN

Project Documentation

Project type	Deep Learning / Computer Vision
Model	Custom Convolutional Neural Network (CNN)
Framework	TensorFlow and Keras
Dataset source	Kaggle: vipoooool/new-plant-diseases-dataset
Notebook environment	Google Colab (as indicated in the notebook)

Prepared from the supplied Jupyter Notebook (Untitled13.ipynb). This document describes the implementation shown in that notebook; measured accuracy and final class counts must be taken from an actual completed run.

Document Guide

- 1. Project overview and objectives
- 2. Problem statement and scope
- 3. Dataset and directory structure
- 4. Tools, libraries, and environment
- 5. System workflow
- 6. Data preparation and augmentation
- 7. CNN model architecture
- 8. Training strategy and callbacks
- 9. Evaluation and interpretation
- 10. Saving and using the model
- 11. Reproducibility and troubleshooting
- 12. Limitations and future improvements
- 13. Conclusion
- Appendix A: Core configuration
- Appendix B: Suggested viva questions

1. Project Overview

Plant Disease Detection using CNN is an image-classification project. It uses a convolutional neural network to learn visual patterns from plant-leaf images and assign an input image to one of the disease/plant categories represented by the training folders. The notebook downloads a Kaggle dataset, discovers the extracted training and validation folders, builds TensorFlow datasets, trains a CNN, evaluates it, and saves the trained model together with the class-name list.

Main objectives

- Prepare a labeled image dataset from the Kaggle archive.
- Automatically identify class folders and create training/validation datasets.
- Train a custom CNN to classify leaf images into the available classes.
- Use augmentation and training callbacks to help generalization and training stability.
- Evaluate performance with validation loss, accuracy, and a classification report.
- Export the model and class labels for later inference or integration.

2. Problem Statement and Scope

Plant diseases can produce visible symptoms on leaves, but inspecting every image manually can be slow and inconsistent. This project explores whether a deep-learning image classifier can learn these visual differences from labeled examples. Its scope is image-level classification: the model predicts one class for an image. It does not draw bounding boxes, segment affected leaf areas, or independently confirm a disease in a field setting.

Expected output

For an input image, the model produces a probability for each class. The class with the highest probability is selected as the predicted label. The model's output is a machine-learning prediction, not a guarantee of an agronomic diagnosis.

3. Dataset and Directory Structure

The notebook specifies the Kaggle dataset identifier **vipoooool/new-plant-diseases-dataset**. It downloads the archive using the Kaggle CLI and extracts it to `/content/plant_dataset`. The notebook then searches the extracted directory tree for folders named train/training, valid/validation/val, and test/testing. Training is stopped if a training directory cannot be found.

The code detects class labels from non-empty subfolders inside the training directory. The exact class count and image count are not hard-coded; they are printed at runtime. Therefore, this documentation does not claim a particular number of images or classes without a completed notebook output.

Expected folder pattern

```
plant_dataset/
  train/
    class_1/
      image_001.jpg
      ...
    class_2/
      ...
  valid/              (if supplied by the dataset)
    class_1/
    class_2/
  test/               (detected, but not used in the shown evaluation)
```

Supported image extensions in the notebook

The directory scanner counts .jpg, .jpeg, .png, .bmp, and .webp files. Class names are derived from folder names and sorted alphabetically. Training and validation class folders must match when a separate validation directory is found.

4. Tools, Libraries, and Environment

Tool / library	Purpose in this project
Python	Main programming language and file/directory handling
Google Colab	Notebook execution environment; Drive is mounted in the first cell
Kaggle CLI	Downloads the dataset archive; Kaggle API access must be configured
TensorFlow / Keras	Image pipeline, CNN construction, training, callbacks, model saving
NumPy	Numerical arrays and random seed setup
Matplotlib	Displays sample images and plots accuracy/loss curves
scikit-learn	Produces the classification report

The notebook seeds Python, NumPy, and TensorFlow with 42 to improve repeatability. Actual results can still vary with hardware, software versions, and execution details.

5. System Workflow

Step	Stage	Description
1	Initialize environment	Import libraries, set random seeds, and check TensorFlow/GPU availability.
2	Download and extract	Download the Kaggle dataset ZIP and extract image files.
3	Discover folders	Find training/validation/test paths and count images.
4	Build datasets	Resize images to 160 × 160, group them into batches of 32, and assign integer labels.
5	Inspect samples	Display up to nine images with their class names.
6	Build CNN	Apply augmentation and rescaling, then convolution/pooling blocks and a classifier head.
7	Train	Fit for up to five epochs with validation data and callbacks.
8	Evaluate	Load the best saved checkpoint and calculate validation metrics/report.
9	Export	Save the final Keras model and JSON class-name file.

6. Data Preparation and Augmentation

Images are loaded using `tf.keras.utils.image_dataset_from_directory`. Each image is resized to 160 × 160 pixels and batches contain 32 images. Labels use integer encoding, which matches the sparse categorical cross-entropy loss used during training.

Validation split behavior

If a validation folder exists, it is loaded separately and its class names are checked against the training class names. If no validation folder exists, the notebook creates an 80% training / 20% validation split from the training directory. The test directory is discovered, but the shown notebook does not evaluate on it; the reported evaluation is on validation data.

Input pipeline optimization

The training and validation datasets use `prefetch(tf.data.AUTOTUNE)` so TensorFlow can overlap input preparation with model execution.

Augmentation transforms

- **RandomFlip(horizontal):** randomly flips training images horizontally.
- **RandomRotation(0.08):** applies small random rotations.
- **RandomZoom(0.10):** applies random zoom changes.
- **RandomContrast(0.10):** varies image contrast.

Augmentation is applied as part of the model and is intended to expose the network to varied versions of training images. The following Rescaling layer converts pixel values from the 0–255 range to approximately 0–1.

7. CNN Model Architecture

The notebook defines a Sequential Keras model with four convolutional blocks. Each block uses a Conv2D layer with ReLU activation, BatchNormalization, and MaxPooling2D. The classifier head uses global average pooling, a dense layer, dropout, and a softmax output.

Layer / block	Configuration	Purpose
Input	160 × 160 × 3	Accepts RGB color images
Augmentation	Flip, rotate, zoom, contrast	Increase variation in training examples
Rescaling	1/255	Normalize pixel intensity
Conv block 1	32 filters, 3×3, same padding, ReLU; BatchNorm; MaxPool	Learn basic edges and textures
Conv block 2	64 filters; BatchNorm; MaxPool	Learn more complex visual patterns
Conv block 3	128 filters; BatchNorm; MaxPool	Build higher-level representations
Conv block 4	256 filters; BatchNorm; MaxPool	Learn richer feature maps
GlobalAveragePooling 2D	Spatial feature averaging	Reduce feature maps to a compact vector
Dense	256 units, ReLU, L2=0.001	Learn combinations of extracted features
Dropout	0.45	Regularization to help reduce overfitting
Output	NUM_CLASSES units, softmax	Produce class probabilities

Important terms

- **Convolution:** scans image regions to detect patterns such as edges, spots, and textures.
- **Pooling:** reduces spatial dimensions and computation.
- **Batch normalization:** normalizes intermediate activations and can stabilize training.
- **ReLU:** adds non-linearity by setting negative activations to zero.
- **Softmax:** converts output scores into a probability distribution across classes.
- **L2 regularization and dropout:** discourage overly complex or overly dependent feature patterns.

8. Training Strategy and Callbacks

The model is compiled with the Adam optimizer at learning rate 0.001, sparse categorical cross-entropy loss, and accuracy as a tracked metric. Training is configured for five epochs in the provided notebook. Five epochs is a short initial run and should not be treated as proof that the model has converged.

Callback	Notebook settings	Purpose
ModelCheckpoint	Monitor val_loss; save best only; mode=min	Preserve the checkpoint with the lowest validation loss
EarlyStopping	Monitor val_loss; patience=5; restore best weights	Stop if validation loss does not improve for several epochs and restore best weights
ReduceLROnPlateau	Monitor val_loss; factor=0.3; patience=2; min_lr=1e-7	Reduce learning rate when validation loss stalls

The best checkpoint is written to `/content/best_plant_disease.keras`. The notebook later reloads that checkpoint before evaluation.

9. Evaluation and Interpretation

The notebook plots training and validation accuracy and loss across epochs. It then loads the best checkpoint and evaluates it on the validation dataset. Finally, it predicts labels for validation batches and prints a scikit-learn classification report.

Metrics included

- **Accuracy:** fraction of predictions that are correct.
- **Loss:** the sparse categorical cross-entropy objective; lower is generally better when comparing the same setup.
- **Precision:** among examples predicted as a class, the fraction that truly belong to that class.
- **Recall:** among true examples of a class, the fraction correctly identified.
- **F1-score:** harmonic mean of precision and recall.
- **Support:** number of validation examples for each class.

No numeric performance result is embedded as a fixed value in the notebook source. The actual validation accuracy, loss, precision, recall, and F1-scores appear only after the notebook is run. Do not report an assumed or advertised accuracy as this run's measured result.

Recommended additional evaluation

- Evaluate once on a held-out test set, if its directory is available, after model decisions are finalized.
- Plot a confusion matrix to inspect which classes are confused.
- Check class-wise precision and recall rather than relying only on overall accuracy.
- Review examples that are misclassified, especially visually similar diseases.

10. Saving and Using the Model

The notebook saves two files:

- `/content/plant_disease_cnn.keras` — the trained Keras model.
- `/content/plant_disease_classes.json` — ordered class names used to map model output indices to readable labels.

Both files are downloaded from Google Colab with `files.download`. Keep the JSON class list with the model: output index 0, 1, and so on correspond to the class names in that exact saved order.

Inference outline

```
import json
import numpy as np
import tensorflow as tf
from PIL import Image

model = tf.keras.models.load_model("plant_disease_cnn.keras")
with open("plant_disease_classes.json", "r") as f:
    class_names = json.load(f)

img = Image.open("leaf.jpg").convert("RGB").resize((160, 160))
x = np.asarray(img, dtype=np.float32)[None, ...]  # shape: (1, 160, 160, 3)

probabilities = model.predict(x, verbose=0)[0]
predicted_index = int(np.argmax(probabilities))
print("Prediction:", class_names[predicted_index])
print("Confidence:", float(probabilities[predicted_index]))
```

Do not divide the inference image by 255 in this outline because the saved model already includes a Rescaling(1/255) layer. If you use a different preprocessing pipeline, it must match the model's expected input

processing.

11. Reproducibility and Troubleshooting

Kaggle download fails or permission denied

Configure Kaggle API credentials in Colab and accept any dataset access requirements. Do not paste API tokens into shared notebooks or public repositories.

Training directory not found

Print the extracted top-level contents and inspect the actual nested path. The notebook expects a training folder containing class subfolders.

Class count is zero or one

Check that the chosen training directory directly contains one folder per class and that those folders contain supported image files.

Training and validation classes do not match

Compare folder names and spelling in both directories. The notebook intentionally stops if the sorted lists differ.

Out-of-memory / slow training

Reduce BATCH_SIZE (for example, 16), confirm a GPU runtime is enabled, or reduce IMG_SIZE if acceptable for the task.

Accuracy does not improve

Check label/folder correctness, class imbalance, image quality, and learning curves. Train longer only after monitoring overfitting and validation performance.

Model file is missing

Confirm training completed and the checkpoint callback wrote the file before loading it.

Predictions map to wrong names

Use the exact JSON file saved with the model; do not recreate class order manually.

12. Limitations and Future Improvements

Limitations of the supplied implementation

- The notebook configures only five training epochs, which may be insufficient for strong performance.
- A test directory is discovered but not used in the shown evaluation; validation metrics alone do not establish final test performance.
- No confusion matrix or sample-level error analysis is included in the supplied notebook.
- The dataset may not represent real-world lighting, camera devices, backgrounds, or field conditions.
- The classifier predicts one of the known folder classes and may still assign a class to an unfamiliar or low-quality image.

Future work

- Run a longer, monitored training schedule and compare validation curves before choosing the final model.
- Use transfer learning (for example, MobileNetV2 or EfficientNet) as a comparison baseline.

- Add a confusion matrix and per-class metric plots.
- Evaluate on a separate test set and, if possible, images collected under real field conditions.
- Consider class balancing or targeted augmentation if some classes are underrepresented.
- Build a simple web/mobile interface that accepts a leaf image and displays the predicted class and confidence.
- Add an uncertainty or 'unknown / please retake image' rule for low-confidence predictions, validated experimentally.

13. Conclusion

The supplied notebook implements an end-to-end plant-leaf image-classification workflow using a custom CNN in TensorFlow/Keras. It covers dataset acquisition, folder discovery, image loading, augmentation, model construction, training callbacks, validation evaluation, classification reporting, and model export. The project provides a foundation for a plant-disease classification prototype. Its actual effectiveness must be established from completed training outputs and, ideally, an independent test set.

Appendix A: Core Configuration

Parameter	Value in notebook
Dataset ID	vipoooool/new-plant-diseases-dataset
Image size	160 × 160 pixels
Batch size	32
Random seed	42
Convolution filters	32 → 64 → 128 → 256
Dense layer	256 units, ReLU, L2 regularization 0.001
Dropout	0.45
Optimizer	Adam, learning rate 0.001
Loss	Sparse categorical cross-entropy
Epochs	5
Batch size / labels	32 / integer labels
Saved model	plant_disease_cnn.keras
Saved class labels	plant_disease_classes.json

Appendix B: Suggested Viva / Presentation Questions

What is the purpose of this project?

To classify plant-leaf images into the classes learned from the labeled dataset.

Why use a CNN for images?

CNNs learn spatial visual features such as edges, textures, and shapes from image pixels.

Why resize images to 160 × 160?

A consistent input size is required by the model and helps manage computational cost.

What does data augmentation do?

It creates varied image transformations during training to help the model generalize.

Why use softmax in the output layer?

It produces a probability distribution over the known classes.

Why sparse categorical cross-entropy?

Labels are integer class IDs rather than one-hot vectors.

What is early stopping?

A callback that stops training when the monitored validation metric fails to improve for a set patience.

Why save class_names.json?

It preserves the mapping from prediction indices to human-readable class names.

Is validation accuracy the same as real-world accuracy?

No. Real-world performance should be checked on an independent test set and representative field images.

What is the most important next improvement?

Complete a robust evaluation with a held-out test set, per-class metrics, and error analysis.